MUC MVP 授權防破解加固方案 v1

0. 文件目的

本文件為獨立文件，專門定義 MUC MVP 階段在「授權防破解」上的設計策略、威脅模型、加固方案、API 驗證機制、離線策略與風險邊界。

本文件**不討論**一般會員系統、收款後台、發票、分潤、代理商、資料回傳、AI 儀表板等功能；只聚焦在：

- 如何避免隨便拿一組序號就能啟動
- 如何降低子機被修改後串去假伺服器模仿啟動的風險
- 如何把破解成本拉高到多數人不願意做

1. 先講結論

1.1 本文件的安全目標

MVP 階段的目標不是「絕對防破解」，而是做到以下五件事：

1. **不能只靠序號就直接啟動。**
2. **不能把 token 複製到另一台設備就直接用。**
3. **不能把 API URL 改到別台伺服器就輕易模仿授權。**
4. **不能在過期或失聯很久後，仍長時間無限制繼續跑。**
5. **就算有人想破解，也需要逆向、patch、假伺服器、重放、簽章偽造等多步驟，成本要高。**

1.2 明確不承諾的事

以下事情在客戶端軟體世界裡，MVP 無法保證：

- 無法保證 100% 不被逆向
- 無法保證 100% 不被 patch
- 無法保證 100% 不被人自己架假伺服器
- 無法保證 100% 不被高手在本機 hook 或修改執行流程

所以本文件的策略是：

不追求絕對無法破解，而是追求可營運、可稽核、可快速發現異常、可提高破解成本。

2. 威脅模型（Threat Model）

2.1 主要攻擊場景

本系統在授權層面要防的，不是單一 bug，而是以下幾類繞過方式：

A. 序號濫用

- 使用者嘗試只拿一組序號，重複啟動多台設備
- 使用者嘗試跳過配對流程，直接把序號當授權憑證使用

B. token 盜用 / 重放

- 使用者從合法裝置把 device_token 取出
- 將 token 複製到另一台設備使用
- 重放合法請求內容，假裝自己是原本那台設備

C. 客戶端 patch

- 修改子機程式，把授權判斷直接改成永遠 ACTIVE
- 直接跳過 entitlement check
- 改成本地固定回傳「可用」

D. 假伺服器模仿

- 將子機的 API 網址改到別台主機
- 自己架一個回應格式相同的 fake server
- 回傳假的 ACTIVE、假的到期日、假的續約資訊

E. 中間人攔截 / 流量篡改

- 攔 HTTPS 流量
- 修改回應內容
- 用 proxy 或 hosts 將請求導到其他端點

F. 離線繞過

- 授權過期後不再連線
- 利用快取狀態讓子機離線繼續跑

2.2 安全假設

以下假設成立時，本文件策略有效；若不成立，需提高安全等級：

- 子機執行在使用者可控制的電腦上
- 使用者可能具備基本逆向能力
- 使用者可能可修改網路設定、hosts、DNS、proxy
- 使用者不一定能成功偽造伺服器私鑰簽章
- 使用者不一定能持續穩定逆向所有版本

3. 安全設計原則

3.1 核心原則一：序號不是授權憑證

序號只能代表「商業上的收費單位」，不能直接成為執行憑證。

也就是說：

- 序號不能直接拿來啟動子機
- 序號不能直接當 entitlement API 的 bearer 憑證
- 序號只能在配對流程中作為關聯資料存在

真正授權鏈應該是：

序號 → 一次性配對碼 → device registration → device token → entitlement → 短效授權票

3.2 核心原則二：核心功能不能只靠一次前置檢查

如果子機只是在啟動時查一次「我可不可用」，之後就一路執行，攻擊者只要 patch 一個 if 判斷就能繞過。

因此：

- 核心工作前必須再驗一次授權
- 授權結果要短效
- 不應只靠本地布林值維持可用狀態

3.3 核心原則三：伺服器回應必須可驗章

如果子機只相信 JSON 內容，而不驗證「這份內容是不是平台真的簽的」，那任何人都可以自架 fake server 模仿回應。

因此：

- entitlement response 必須有伺服器簽章
- 子機必須內建平台公鑰驗章
- 無法驗章的回應，應視為不可信

3.4 核心原則四：離線只能有限容忍

只要軟體在客戶端執行，就無法完全禁止離線運作；但可以限制離線可用時間。

因此：

- 可短時間容忍 API 失敗
- 不可長時間離線無限制繼續跑
- 超過門檻後應保守停用或停止接新任務

4. 授權鏈路設計（License Chain of Trust）

4.1 配對前狀態

配對前，子機不得具備正式可用授權。

可允許：

- 安裝
- 基本設定
- 顯示未啟用畫面

不可允許：

- 執行正式工作
- 接新任務
- 啟動完整核心流程

4.2 一次性配對碼（Pair Code）

配對碼只用於第一段配對，不可當長期授權憑證。

規則

- 一次性使用
- 短時效，例如 5～10 分鐘
- 綁定特定 account / license
- 使用成功後立刻作廢
- 伺服器只存 hash，不存明文
- 配對失敗過多次需失效

風險控制

- pair code 洩漏的風險只能控制在短時窗內
- 成功配對後不可重複使用同一碼

4.3 設備註冊（Device Registration）

配對成功時，伺服器建立正式設備紀錄。

伺服器端建立資料

- device_id
- license_id
- account_id
- fingerprint_hash
- device_token_hash
- registered_at
- status=ACTIVE

規則

- 一個序號只能綁一個 ACTIVE device
- 若已有 ACTIVE device，再綁新設備時需明確取代或先 revoke 舊設備
- 同一 device_token 不可共用多個 device record

4.4 device_token

device_token 是設備後續查授權的長期憑證，但它本身不應等於「無條件執行權」。

規則

- 高熵亂數產生
- 僅首次註冊成功時回傳一次明文
- 伺服器只存 hash
- 可被 revoke
- 可設定版本 / 失效時間 / 輪替策略

儲存策略

- 客戶端：OS 安全儲存（Keychain / Credential Manager / Secret Store）
- 伺服器：只存 hash，不存明文

4.5 fingerprint 綁定

fingerprint 的目的不是絕對證明「就是這台硬體」，而是提高複製與換機成本。

建議來源：

install_id + platform + app_version + os_version + optional hardware hints → SHA-256

使用原則

- fingerprint 用於風險判斷與設備一致性檢查
- 不把 fingerprint 當成唯一不可偽造的硬證據
- 不一致時可要求重新配對

不建議過度依賴的原因

- install_id 可被複製
- OS / app version 可被偽造
- hardware hints 在不同 OS 下可靠度不一致

5. Entitlement API 加固設計

5.1 基本請求條件

子機呼叫 entitlement API 時，至少帶：

- Authorization: Bearer <device_token>
- X-Device-Id
- X-Fingerprint
- X-App-Version
- X-Request-Nonce
- X-Request-Timestamp

5.2 伺服器驗證步驟

伺服器處理 entitlement 時，至少驗以下幾層：

1. token hash 是否存在
2. token 是否已 revoke / expired
3. token 對應的 device_id 是否一致
4. device 狀態是否為 ACTIVE
5. device 綁定的 license / subscription 是否可用
6. fingerprint 是否符合可接受範圍
7. request timestamp 是否過舊
8. nonce 是否重複（避免重放）
9. request 是否超過 rate limit

5.3 entitlement response 不能只回裸 JSON

錯誤示範：

```
{
  "status": "ACTIVE",
  "expires_at": "2026-03-31T23:59:59+08:00"
}
```

這種格式很容易被假伺服器模仿。

正確做法是：

- 回傳 payload
- 回傳 issued_at
- 回傳 expires_at
- 回傳 signature
- 子機本地用平台公鑰驗章

5.4 建議 response 結構


```
{
  "payload": {
    "device_id": "dev_xxx",
    "license_id": "lic_xxx",
    "subscription_status": "ACTIVE",
    "allow_new_jobs": true,
    "grace_until": null,
    "message_to_user": "",
    "message_to_agent": "",
    "job_grant_ttl_seconds": 180
  },
  "issued_at": "2026-03-10T12:00:00+08:00",
  "expires_at": "2026-03-10T12:03:00+08:00",
  "signature": "base64(signature)"
}
```


5.5 子機本地驗證規則

子機收到 response 後，必須驗以下項目：

1. issued_at / expires_at 是否在可接受範圍
2. payload.device_id 是否等於本機 device_id
3. signature 是否可被平台公鑰驗證
4. payload 是否未過期
5. allow_new_jobs 是否為 true

只要其中一項失敗，該回應就不可信。

6. 短效授權票（Job Grant）

6.1 為什麼需要 Job Grant

如果子機只靠「上一次 entitlement check 是 ACTIVE」就能一直跑，那 patch 一個布林判斷就有機會繞過。

因此，建議把「接新任務」這件事改成要依賴一張**短效授權票**。

6.2 Job Grant 的用途

Job Grant 代表：

伺服器在某個極短時間窗內，同意這台合法設備接一個或一批新任務。

6.3 Job Grant 建議欄位

- grant_id
- device_id
- license_id
- issued_at
- expires_at
- nonce
- scope（例如 accept_new_jobs）
- signature

6.4 使用方式

- entitlement API 回應時一併附帶短效 job grant，或提供專用 grant API
- grant TTL 建議 1～5 分鐘
- 每次接新任務前必須確認 grant 未過期
- grant 驗章失敗或過期時，不可接新任務

6.5 安全意義

這樣攻擊者不能只 patch 一次前置檢查，而是要能持續偽造：

- 正確 payload
- 正確 device_id
- 正確 expires_at
- 正確 signature

破解難度會明顯上升。

7. 防假伺服器策略

7.1 Certificate Pinning

子機應內建平台 API 憑證或公鑰 pin，避免被任意 HTTPS 端點冒充。

目標

降低以下風險：

- hosts 被改
- DNS 被改
- proxy 導流
- 自簽或替代憑證被接受

注意

- pinning 不是萬靈丹，仍可能被進階 hook 繞過
- 但可擋掉大多數「改個網址就接假 server」的低階攻擊

7.2 Response Signing

就算有 pinning，仍建議每份 entitlement / job grant 都加簽。

原因：

- pinning 是「你連的是誰」
- signing 是「你收到的內容是不是平台真的簽的」

兩者缺一不可。

7.3 API Base URL 防修改

建議：

- base URL 不做成明文可任意改 config
- 若需多環境，可限制成 build-time 或 signed config
- 不提供一般使用者可見的自訂授權伺服器欄位

8. 防重放策略

8.1 Request Nonce

每次 entitlement 請求都要帶 nonce，伺服器短時間內不可接受重複 nonce。

8.2 Timestamp Window

每次請求都要帶時間戳，伺服器只接受合理時間窗，例如 ±60 秒。

8.3 短效回應

entitlement 與 job grant 都應短效，例如 1～5 分鐘，不應給過長可用期。

8.4 Session 綁定

必要時可將 grant 與：

- device_id
- token id
- app version
- request nonce
  綁在一起，降低重放價值。

9. 離線策略

9.1 原則

MVP 可以容忍短時間斷線，但不能容忍長時間離線照常跑。

9.2 建議規則

- 每 2～3 分鐘輪詢一次 entitlement
- 每次接新任務前必查一次
- API 失敗累計 ≤ 15 分鐘：沿用最後一次合法狀態
- API 失敗累計 > 15 分鐘：停止接新任務
- 已在執行中的任務可完成，但不得再接新任務

9.3 離線快取條件

只有在以下條件都成立時，才可沿用上一份快取授權：

1. 上一次 response 有合法簽章
2. response 尚未過離線容忍上限
3. 本機未偵測到 token / fingerprint / app version 異常
4. 設備未進入 revoke 標記的可疑狀態

9.4 過期策略

一旦 subscription 已過期，且離線又超過容忍門檻，不得再接新任務。

10. Token 複製與換機控制

10.1 Token 複製風險

只要 token 落到其他人手上，就有被複製的可能。

因此不能只靠 token 本身，要再綁：

- device_id
- fingerprint
- recent behavior

10.2 異常偵測規則

以下情境應觸發告警或直接鎖定：

- 同一 token 在短時間內來自不同 device_id
- 同一 token 在短時間內來自不同 IP / 地理位置
- fingerprint 頻繁變動
- 同一 license 在短時間內切換多個設備

10.3 換機規則

建議 MVP 明定：

- 一個序號同時只允許一台 ACTIVE 設備
- 換機時，需先 revoke 舊設備或走取代流程
- 新設備一定要重新配對，不能直接複製 token 生效

11. 子機端反竄改建議（MVP 級）

11.1 可做但不要過度承諾

MVP 可做基本反竄改，但不應承諾「一定抓得到所有 patch」。

11.2 建議項目

- 基本 binary integrity check
- 關鍵授權流程混淆（obfuscation）
- 關鍵字串避免明文寫死
- 防常見 debug / hook 工具的偵測
- 版本檢查與最低版本限制

11.3 原則

這些措施的目的不是絕對阻止高手，而是提高維護破解版本的成本。

12. 稽核與風控

12.1 必記錄事件

以下事件應記錄到不可刪 audit log：

- pair code 產生 / 使用 / 失敗
- device 註冊成功 / 失敗
- token 發行 / revoke / rotation
- entitlement 驗證失敗
- fingerprint mismatch
- nonce replay
- rate limit hit
- suspicious IP / geolocation anomaly
- license 被暫停 / 恢復

12.2 告警級別建議

- INFO：正常配對、正常續期
- WARN：fingerprint 改變、頻繁失敗、可疑切換
- CRITICAL：token 重放、假伺服器疑慮、簽章驗證失敗、同 token 多設備使用

12.3 風險處置策略

依事件類型可採：

- 僅告警
- 要求重新配對
- 暫停接新任務
- 直接 revoke token
- 暫停該序號

13. API 設計建議（授權防破解版）

13.1 Pair API

POST /api/device/pair

用途

- 使用一次性配對碼完成設備註冊
- 回傳 device token 與初始 entitlement 資訊

伺服器需驗證

- pair code hash
- pair code 是否過期
- pair code 是否已使用
- 該 license 是否允許綁定新設備

13.2 Entitlement API

GET /api/device/entitlement

用途

- 提供短效、可驗章的授權資訊
- 必要時附帶 job grant

回應要求

- payload
- issued_at
- expires_at
- signature

13.3 Pre-Job Check API

POST /api/device/check-before-job

用途

- 在接新任務前再做一次授權確認
- 回傳或更新短效 job grant

13.4 Revoke API（平台端）

POST /admin/devices/{id}/revoke

用途

- 平台或使用者主動停用設備
- revoke 後 token 立即失效

14. MVP 最低安全基線（Minimum Security Baseline）

如果只能做最必要的加固，最低至少要有以下 8 項：

1. **序號不能直接當授權憑證**
2. **一次性配對碼 + 短時效 + 只存 hash**
3. **device_token 高熵亂數 + 只存 hash + 可 revoke**
4. **token 綁 device_id 與 fingerprint 檢查**
5. **entitlement response 一律加簽**
6. **子機內建公鑰驗章**
7. **每次接新任務前必查授權或 grant**
8. **離線超過 15 分鐘後停止接新任務**

若這 8 項沒有同時做到，就很難說這套授權有基本防繞過能力。

15. 建議的加固優先順序

P1：MVP 必做

- pair code 一次性 / 短時效 / hash
- device token hash 儲存
- token revoke 機制
- entitlement response signing
- 子機公鑰驗章
- 每次接新任務前必查授權
- 離線容忍上限
- audit log 與異常告警

P2：MVP 強化版

- job grant
- certificate pinning
- nonce replay 防護
- timestamp window 驗證
- base URL 防任意修改

P3：後續進階版

- binary integrity 深度檢查
- anti-debug / anti-hook 強化
- 更細的風險評分
- 自動封鎖策略
- 更完整的裝置指紋與行為分析

16. 風險邊界與誠實說明

16.1 可以達成的效果

若本文件設計落地，MVP 可達到：

- 正常使用者無法只靠序號直接啟動
- token 複製到另一台設備的成功率會下降
- 改 API URL 指到假伺服器的低階模仿會被大幅阻擋
- 長時間離線無限制運作會被抑制
- 大多數破解者需要投入明顯更高成本

16.2 仍無法保證的事

即便全部做完，仍不能保證：

- 不會被進階逆向
- 不會被 runtime hook
- 不會被 patch 掉本地驗章流程
- 不會被高手維護私有破解版本

因此，本文件的真正目標應理解為：

把破解從「隨便改一下就能用」提升到「需要專業逆向能力且維護成本高」。

17. 結論

MUC MVP 的授權防破解，不應只做「序號 + token + 輪詢」，而應升級成一條完整授權信任鏈：

序號 → 配對碼 → 設備註冊 → device token → entitlement signing → 短效 job grant → 離線上限 → 稽核告警（這才是 MVP 階段最務實、最可落地的授權防破解方案。）

這樣做的價值不是讓系統變成絕對無法破解，而是讓：

- 一般人無法亂用
- 假伺服器不好模仿
- token 複製不好重放
- 子機 patch 不再只改一行就過
- 平台能追蹤、發現、封鎖異常使用